

# Parallel and Distributed Systems

Exercise 3: Problems and How Parallelize Them

Prof. Dr. Martin Sträßer



# Task

- In the following, we will discuss three different problems. Every problem could profit from a parallelized solution to increase throughput and reduce runtime
- Your task is to think about the problems and identify opportunities for parallelization (e.g., via multiple processes)
- Which factors would impact how far we can parallelize?
- Use terms from the lecture such as data parallelism and task parallelism correctly



# Problem 1: Image Processing

- You are building a system for a photo-sharing platform that needs to apply filters (resize, color correction, blur, edge detection) to **millions of images** uploaded every day
- This is a good **data parallelism** use case, we can distribute images across processes, optionally split each image into tiles
- We could also build a **pipeline**



## Problem 2: Smart Home

- You are designing a smart home system that continuously:
  - Monitors temperature sensors
  - Detects motion from cameras
  - Controls lighting
  - Sends alerts to users
  - Logs data to a database
- This is a good **task parallelism** use case, we can have separate processes for sensor monitoring and video processing
- Processes a good fit for fault isolation



## Problem 3: Video Streaming

- You are implementing a video streaming service that processes uploaded videos through multiple stages:
  - Upload video
  - Decode raw frames
  - Apply compression
  - Generate thumbnails
  - Store and distribute
- This is a good use case for **pipeline parallelism**, each stage runs in parallel on different data chunks

# 5 Quick Questions About Processes

1. Name four components of the memory space of a process
2. Besides edge cases, two processes have independent address spaces. True or false?
3. A child process is typically executed after its parent process. True or false?
4. We are executing many processes and run low on main memory (RAM). Name two options the operating system has to handle this situation.
5. Why does copy-on-write (COW) speed up process creation?



# Solutions

1. Stack, heap, memory mappings, program code, environment/global variables, ...
2. This is true. Only immediately after forking, there are some dependencies between processes.
3. This is false. We do not control the execution order/scheduling, this is the job of the operating system.
4. The OS could kill processes with huge memory demand (e.g., Linux OOM killer). Also it might “swap out” processes to the disk memory.
5. Because it is faster than full copies of the memory. Full copies are only done if required.